Teach Me Skills

Lombok и логирование

Lombok

Lombok — проект по добавлению дополнительной функциональности в Java с помощью изменения исходного кода перед Java компиляцией.

По сути, проект Lombok позволяет избавиться от многословности Java в большинстве случаев и перестать писать огромные простыни кода из гетеров, сеттеров, equals, hashcode и toString (да их обычно генерит IDE, но читать и менять все равно приходится программисту), в результате Java становиться почти такой же краткой как Kotlin, Scala или C#.

teachmeskills.by

Lombok - использование

- @Getter/@Setter – добавление Getter и Setter в класс
- @ToString – добавление toString в класс
- @EqualsAndHashCode – генерация equals и hashCode функций
- @NoArgsConstructor and @AllArgsConstructor – генерация всех типов конструкторов
- @Data – генерация getter, setter, toString, equals и hashCode функций
- @Builder – имплементация Builder Pattern
- @Log4j2 – добавление логгера без лишних проблем (нужна доп.зависимость)

teachmeskills.by

```
<!-- https://mvnrepository.com/artifact/org.projectlombok/lombok -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.8</version>
    <scope>provided</scope>
</dependency>
```

Логирование

Логи - это записи в консоли либо файле, которые позволяют узнать больше информации о происходящем во фреймворке во время выполнения тестов. Их ключевая функция - визуализация того, как тест выполнялся на удаленном сервере (там мы экрана/браузера в большинстве случаев не видим)

Что логировать? - Все по максимуму, чтобы понимать, что происходило в тесте не видя браузера

Что не логировать? - Избегать логов со статичным контентом, лучше всегда логировать какие-либо переменные

teachmeskills.by

Логирование. Lombok и Log4J

- Для логирования настоятельно НЕ рекомендуется использовать System.out.println. Используйте специальные библиотеки, например log4j. Такие библиотеки позволяют логировать одновременно и в консоль и в файл
- Логи могут иметь разные уровни, подробнее:
  - fatal
  - error
  - warn
  - info
  - debug
  - trace

```
<!--
https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.14.1</version>
</dependency>

<!--Для считывания yaml-файлов -->
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-yaml</artifactId>
  <version>2.12.3</version>
</dependency>
```

teachmeskills.by

Конфигурация уровня логирования

- Файл log4j2-test.yaml, который лежит в src/test/resources
- Пример с логированием и в консоль и в файл: https://gist.github.com/dzmitryrak/3a0ade73a92623c1ebd0326819f5d3ab

```yaml
#https://springframework.guru/log4j-2-configuration-using-yaml/
#https://logging.apache.org/log4j/2.x/manual/configuration.html
Configuration:
 status: INFO
 Properties:
   Property:
     - name: "DefaultPattern"
       value: "%d{yyyy-MM-dd HH:mm:ss.SSS} %level [%t] [%c] - %msg%n"
 Appenders:
   Console:
       name: consoleAppender
       target: SYSTEM_OUT
       PatternLayout:
         Pattern: ${DefaultPattern}
 Loggers:
   logger:
     -
       name: consoleLogger
       level: INFO
       additivity: true
       AppenderRef:
         ref: consoleAppender
   Root:
     level: INFO
     AppenderRef:
      - ref: consoleAppender
        level: INFO
```

teachmeskills.by